手写小GPT学习笔记

一、掩码机制的作用

在 Transformer 架构中，掩码机制用于在注意力计算过程中屏蔽不需要关注的位置，确保模型只处理有效的信息，主要分为两种类型：**因果掩码（Causal Mask）和填充掩码（Padding Mask）**。

二、因果掩码（Causal Mask）

1. 原理

因果掩码也称为前瞻掩码（Look-Ahead Mask），主要用于解码器的自注意力机制中，目的是防止模型在生成序列时 "偷看" 未来的 token，确保当前位置的预测仅依赖于该位置之前的 token，保证序列生成的因果性。

在注意力分数矩阵中，因果掩码会将**当前位置之后的所有位置**的分数设置为负无穷，经过 softmax 后这些位置的权重会趋近于 0，从而模型无法获取未来 token 的信息。

2. 实现代码

Code block

```
import torch
import torch.nn as nn
import math

def create_causal_mask(seq_len, device=None):
    """
    创建因果掩码
    Args:
        seq_len: 序列长度
        device: 设备
    Returns:
        因果掩码，形状为[1, seq_len, seq_len]，布尔类型，True表示需要屏蔽的位置
    """
    # 创建上三角矩阵，diagonal=1表示主对角线以上的元素为True（未来位置）
    mask = torch.triu(torch.ones(seq_len, seq_len, device=device), diagonal=1).bool()
    # 扩展batch维度，方便广播
    return mask.unsqueeze(0)

# 测试因果掩码
if __name__ == "__main__":
```

```
    seq_len = 5
    causal_mask = create_causal_mask(seq_len)
    print("因果掩码形状:", causal_mask.shape)
    print("因果掩码内容:")
    print(causal_mask)
```

3. 在注意力机制中的使用

Code block

```
def apply_causal_mask(scores, causal_mask):
    """
    对注意力分数应用因果掩码
    Args:
        scores: 注意力分数矩阵，形状[batch_size, seq_len_q, seq_len_k]
        causal_mask: 因果掩码，形状[1, seq_len, seq_len]
    Returns:
        掩码后的注意力分数
    """
    # 将掩码位置的分数设置为负无穷
    return scores.masked_fill(causal_mask, float('-inf'))
```

三、填充掩码（Padding Mask）

1. 原理

填充掩码用于处理变长序列，当不同长度的序列被打包成一个 batch 时，会使用 PAD token 填充到相同长度。填充掩码的作用是屏蔽这些 PAD token 的位置，防止模型将填充的无意义信息作为有效信息处理。

在注意力分数矩阵中，填充掩码会将**PAD token 对应的位置**的分数设置为负无穷，经过 softmax 后这些位置的权重会趋近于 0。

2. 实现代码

Code block

```
def create_padding_mask(seq, pad_idx):
    """
    创建填充掩码
    Args:
        seq: 输入序列，形状[batch_size, seq_len]
        pad_idx: PAD token的索引
    Returns:
        填充掩码，形状[batch_size, 1, seq_len]，布尔类型，True表示需要屏蔽的位置
```

```
    """
    # 生成掩码：seq == pad_idx的位置为True（需要屏蔽）
    mask = (seq == pad_idx).unsqueeze(1)
    # 扩展维度，方便广播到注意力分数矩阵[batch_size, seq_len_q, seq_len_k]
    return mask

# 测试填充掩码
if __name__ == "__main__":
    # 模拟输入：batch_size=2, seq_len=5, pad_idx=0
    seq = torch.tensor([[1, 2, 3, 0, 0], [4, 5, 0, 0, 0]])
    pad_idx = 0
    padding_mask = create_padding_mask(seq, pad_idx)
    print("填充掩码形状:", padding_mask.shape)
    print("填充掩码内容:")
    print(padding_mask)
```

3. 适配注意力分数矩阵的填充掩码

在自注意力中，注意力分数矩阵的形状是 `[batch_size, seq_len, seq_len]`，需要将填充掩码扩展为 `[batch_size, seq_len, seq_len]` 的形状：

Code block

```
def create_attn_padding_mask(seq, pad_idx):
    """
    创建适用于注意力分数矩阵的填充掩码
    Args:
        seq: 输入序列，形状[batch_size, seq_len]
        pad_idx: PAD token的索引
    Returns:
        填充掩码，形状[batch_size, seq_len, seq_len]，布尔类型，True表示需要屏蔽的位置
    """
    batch_size, seq_len = seq.shape
    # 生成基础掩码 [batch_size, 1, seq_len]
    mask = (seq == pad_idx).unsqueeze(1)
    # 扩展为 [batch_size, seq_len, seq_len]
    return mask.expand(batch_size, seq_len, seq_len)
```

四、两种掩码的结合使用

在解码器的自注意力中，需要同时使用因果掩码和填充掩码，既屏蔽未来 token，又屏蔽 PAD token：

Code block

```python
def combine_masks(causal_mask, padding_mask):
    """
    结合因果掩码和填充掩码
    Args:
        causal_mask: 因果掩码，形状[1, seq_len, seq_len]
        padding_mask: 填充掩码，形状[batch_size, seq_len, seq_len]
    Returns:
        组合后的掩码，形状[batch_size, seq_len, seq_len]
    """
    # 两个掩码取或，只要有一个为True就需要屏蔽
    return torch.logical_or(causal_mask, padding_mask)

# 测试组合掩码
if __name__ == "__main__":
    seq = torch.tensor([[1, 2, 3, 0, 0], [4, 5, 0, 0, 0]])
    pad_idx = 0
    seq_len = seq.shape[1]

    causal_mask = create_causal_mask(seq_len)
    padding_mask = create_attn_padding_mask(seq, pad_idx)
    combined_mask = combine_masks(causal_mask, padding_mask)

    print("组合掩码形状:", combined_mask.shape)
    print("组合掩码内容:")
    print(combined_mask)
```

五、在注意力机制中使用掩码

将掩码应用到之前实现的 Scaled Dot-Product Attention 中：

Code block

```python
class ScaledDotProductAttention(nn.Module):
    def __init__(self, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

    def forward(self, q, k, v, mask=None):
        d_k = q.size(-1)
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)

        # 应用掩码
        if mask is not None:
            scores = scores.masked_fill(mask, float('-inf'))

        attn_weights = self.dropout(torch.softmax(scores, dim=-1))
        output = torch.matmul(attn_weights, v)
```

```python

        return output, attn_weights

# 测试带掩码的注意力机制
if __name__ == "__main__":
    batch_size = 2
    seq_len = 5
    d_model = 64
    pad_idx = 0

    # 模拟输入
    q = torch.randn(batch_size, seq_len, d_model)
    k = torch.randn(batch_size, seq_len, d_model)
    v = torch.randn(batch_size, seq_len, d_model)
    seq = torch.tensor([[1, 2, 3, 0, 0], [4, 5, 0, 0, 0]])

    # 创建掩码
    causal_mask = create_causal_mask(seq_len)
    padding_mask = create_attn_padding_mask(seq, pad_idx)
    combined_mask = combine_masks(causal_mask, padding_mask)

    # 初始化注意力模块
    attn = ScaledDotProductAttention(dropout=0.1)

    # 前向传播
    output, attn_weights = attn(q, k, v, mask=combined_mask)

    print(f"输出形状: {output.shape}")
    print(f"注意力权重形状: {attn_weights.shape}")
    print("注意力权重（第一样本）:")
    print(attn_weights[0])
    # 验证掩码效果：PAD位置和未来位置的权重应该接近0
    print("注意力权重求和（第一样本，每行和为1）:")
    print(attn_weights[0].sum(dim=-1))
```

六、关键注意事项

1. **掩码形状**：因果掩码的形状通常是 `[1, seq_len, seq_len]`，填充掩码的形状通常是 `[batch_size, 1, seq_len]`，需要确保掩码能广播到注意力分数矩阵的形状 `[batch_size, seq_len_q, seq_len_k]`
2. **掩码类型**：布尔类型的掩码中，`True` 表示需要屏蔽的位置，会被设置为负无穷；也可以使用数值类型的掩码，直接加到注意力分数上
3. **解码器的掩码**：解码器的自注意力需要同时使用因果掩码和填充掩码，而解码器对编码器输出的交叉注意力只需要使用填充掩码（屏蔽编码器的 PAD token）

4. **数值稳定性**：使用 `-1e9` 作为负无穷的值，而不是 `float('-inf')` ，避免在某些计算中出现数值问题

（注：文档部分内容可能由 AI 生成）